

Nombres: Richard Steven

Apellidos: Arizandieta Yol

Carné: 202400111

Curso: (LAB) ORGANIZACION DE LENGUAJES Y COMPILADORES 1 - Sección C

Catedrático: Kevin Adiel Lajpop Ajpacaja

Auxiliar: Johnny Whillman Aldana Osorio



MANUAL TÉCNICO PARA MANIPULACIÓN DE PROYECTO 2

INTÉRPRETE DE LENGUAJE GOSCRIP

GUATEMALA 12 DE ABRIL DE 2026

Introducción

El presente manual técnico documenta el desarrollo del sistema GoScript, una aplicación que implementa un intérprete para un lenguaje de programación basado en una sintaxis simplificada inspirada en Go. Este proyecto fue desarrollado en el curso Organización de Lenguajes y Compiladores 1, con el objetivo de aplicar los conceptos de análisis léxico, sintáctico y construcción de estructuras como el Árbol de Sintaxis Abstracta (AST).

El sistema permite:

- Interpretar código fuente con extensión “.gst”.
- Generación de reportes que incluyen errores, tabla de símbolos y representación del AST

Tecnologías Utilizadas

Lenguaje de Programación:

- JavaScript

Herramientas de Compiladores:

- Jison (Generador de análisis léxico y sintáctico)

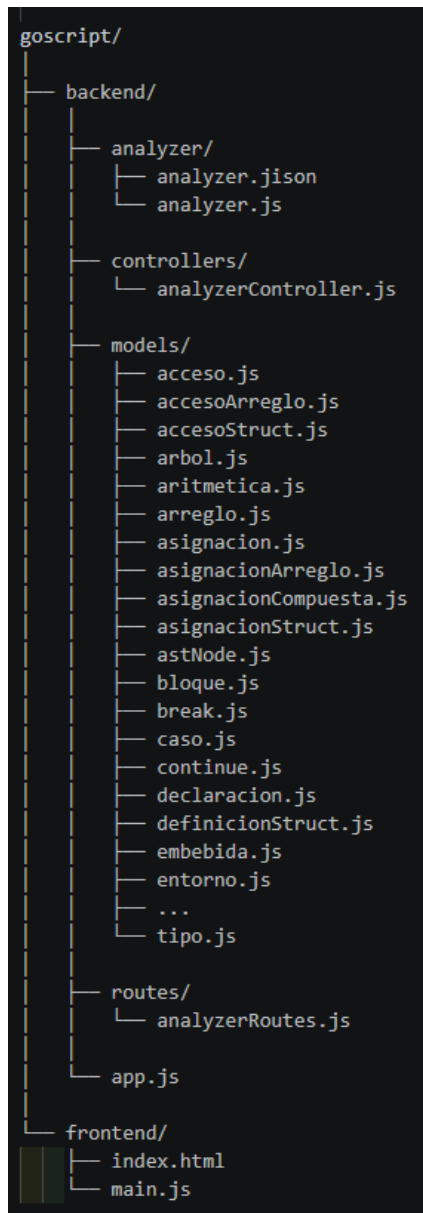
Interfaz gráfica:

- Node.js

Formato de Persistencia:

- Aplicación web (frontend)
-

Arquitectura del sistema



Descripción de módulos clave:

Analyzer: Contiene la definición del analizador léxico y sintáctico implementado conJison, así como su versión en JS generada a partir de la compilación de la gramática.

Controllers: Incluye el controlador principal (AnalyzerController), encargado de recibir el código fuente desde el frontend, enviarlo al analizador y retornar los resultados obtenidos.

Models: Contiene las estructuras fundamentales del lenguaje, incluyendo nodos del AST, expresiones, instrucciones y estructuras de control como asignaciones, ciclos, condicionales y funciones. Estas permiten representar y ejecutar el código interpretado.

Routes: Aquí se definen las rutas del sistema, que permiten la comunicación entre el frontend y el backend mediante solicitudes HTTP.

App: Archivo principal del backend, encargado de configurar el server, registrar las rutas y gestionar la ejecución de la aplicación.

Descripción de Archivos Clave:

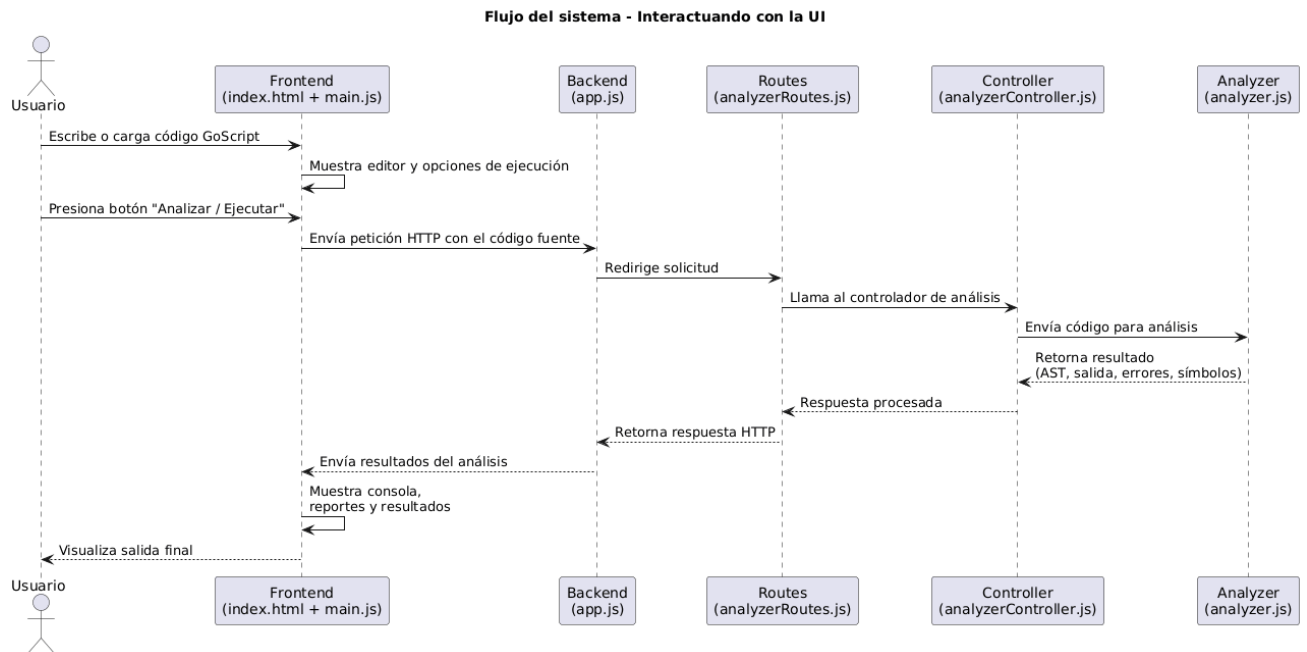
Archivo	Ubicación	Función principal	Descripción
<i>app.js</i>	/backend	Configuración del servidor	Archivo principal del backend. Inicializa el servidor, configura middlewares y registra las rutas del sistema.
<i>analyzer.jison</i>	/backend/analyzer	Definición del analizador	Contiene la gramática del lenguaje GoScript, incluyendo reglas léxicas y sintácticas utilizadas para generar el parser.
<i>analyzer.js</i>	/backend/analyzer	Analizador generado	Archivo generado automáticamente por Jison a partir de la gramática. Se encarga de realizar el análisis léxico y sintáctico del código fuente.
<i>analyzerController.js</i>	/backend/controllers	Controlador de análisis	Recibe el código enviado desde el frontend, ejecuta el analizador y retorna los resultados (AST, errores, salida).
<i>analyzerRoutes.js</i>	/backend/routes	Definición de rutas	Define los endpoints HTTP para la comunicación entre el frontend y el backend.
<i>index.html</i>	/frontend	Interfaz de usuario	Estructura principal de la aplicación web donde el usuario ingresa el código y visualiza resultados.

<i>main.js</i>	/frontend	Lógica del cliente	Gestiona la interacción con el usuario, envía el código al backend y muestra los resultados en la interfaz.
<i>arbol.js</i>	/backend/models	Representación del AST	Define la estructura del Árbol de Sintaxis Abstracta y permite su recorrido durante la ejecución.
<i>astNode.js</i>	/backend/models	Nodo base del AST	Clase base para todos los nodos del AST, utilizada para representar expresiones e instrucciones.
<i>entorno.js</i>	/backend/models	Entorno de ejecución	Administra la tabla de símbolos, variables, funciones y el contexto durante la ejecución del programa.
<i>simbolo.js</i>	/backend/models	Representación de símbolo	Define la estructura de un símbolo dentro de la tabla (nombre, tipo, valor, entorno).
<i>tipo.js</i>	/backend/models	Tipos de datos	Define los tipos de datos soportados por el lenguaje GoScript.
<i>literal.js</i>	/backend/models	Valores primitivos	Representa valores constantes como enteros, cadenas, booleanos, entre otros.
<i>aritmetica.js</i>	/backend/models	Operaciones aritméticas	Implementa la evaluación de operaciones como suma, resta, multiplicación y división.
<i>logica.js</i>	/backend/models	Operaciones lógicas	Maneja operaciones booleanas como AND, OR y NOT.
<i>relacional.js</i>	/backend/models	Operaciones relacionales	Evalúa comparaciones como ==, !=, <, >, <=, >=.
<i>declaracion.js</i>	/backend/models	Declaración de variables	Maneja la creación de variables dentro del entorno de ejecución.
<i>asignacion.js</i>	/backend/models	Asignación de valores	Permite actualizar el valor de una variable existente.
<i>funcion.js</i>	/backend/models	Definición de funciones	Representa la declaración y ejecución de funciones dentro del lenguaje.

<i>llamada.js</i>	/backend/models	Llamada a funciones	Ejecuta funciones previamente definidas, evaluando sus parámetros.
<i>sentenciaIf.js</i>	/backend/models	Estructura condicional	Implementa la lógica de la sentencia if-else.
<i>sentenciaSwitch.js</i>	/backend/models	Estructura switch	Maneja múltiples casos de evaluación condicional.
<i>sentenciaFor.js</i>	/backend/models	Ciclo for	Implementa ciclos de repetición tipo for.
<i>sentenciaWhile.js</i>	/backend/models	Ciclo while	Implementa ciclos de repetición basados en condición.
<i>break.js</i>	/backend/models	Control de flujo	Permite salir de ciclos o estructuras de control.
<i>continue.js</i>	/backend/models	Control de flujo	Permite continuar con la siguiente iteración de un ciclo.
<i>return.js</i>	/backend/models	Retorno de funciones	Permite devolver valores desde una función.
<i>print.js</i>	/backend/models	Salida de datos	Implementa la impresión de valores en la consola del sistema.
<i>arreglo.js</i>	/backend/models	Estructura de datos	Representa arreglos o listas dentro del lenguaje.
<i>accesoArreglo.js</i>	/backend/models	Acceso a arreglos	Permite acceder a posiciones específicas dentro de un arreglo.
<i>nativasSlice.js</i>	/backend/models	Funciones nativas	Implementa funciones propias del lenguaje para manejo de slices.
<i>definicionStruct.js</i>	/backend/models	Definición de struct	Permite definir estructuras de datos personalizadas.
<i>instanciaStruct.js</i>	/backend/models	Instancia de struct	Crea instancias de estructuras definidas por el usuario.
<i>accesoStruct.js</i>	/backend/models	Acceso a struct	Permite acceder a atributos de estructuras.
<i>excepcion.js</i>	/backend/models	Manejo de errores	Representa errores generados durante la ejecución del programa.

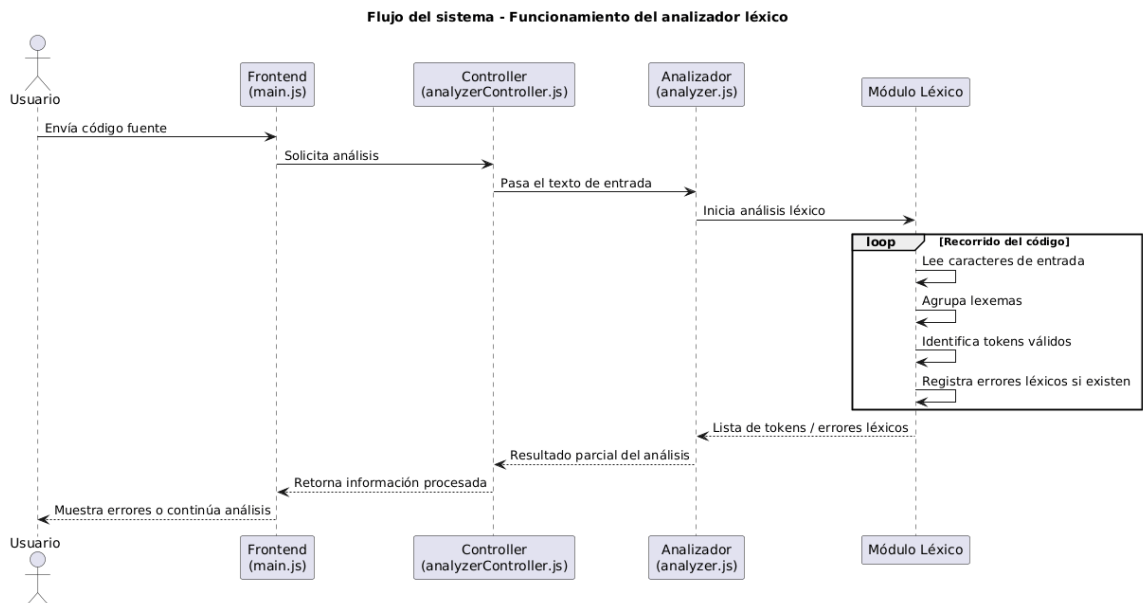
Flujo de Datos

- Interactuando con la UI



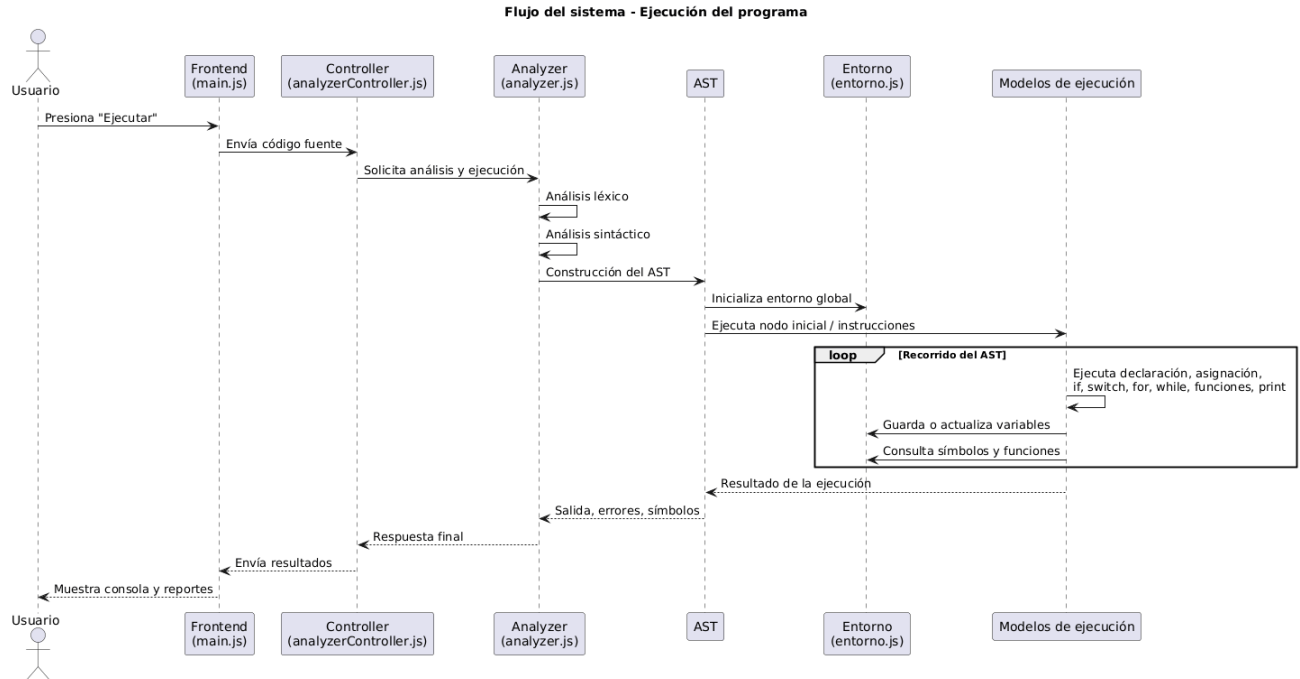
[Ver imagen en grande](#)

- Funcionamiento del analizador léxico



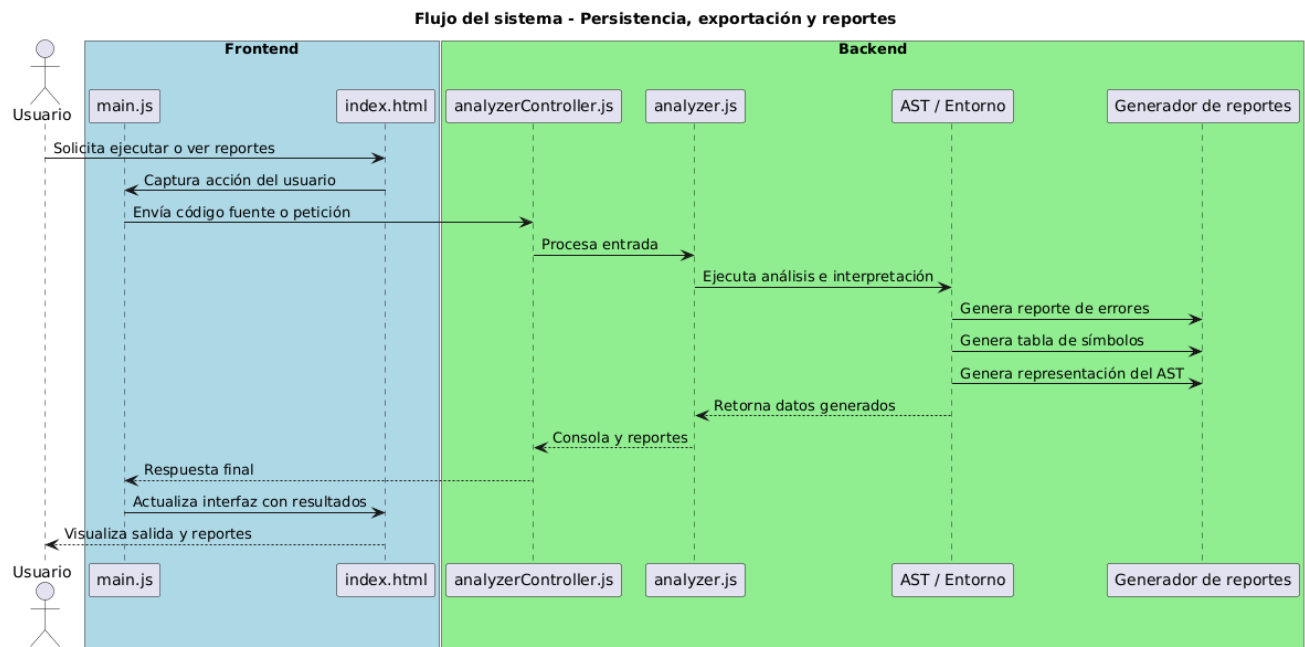
[Ver imagen en grande](#)

- Ejecución del programa



[Ver imagen en grande](#)

- Persistencia, exportación y reportes



[Ver imagen en grande](#)

Decisiones Técnicas y Limitaciones

Decisiones Técnicas

1. Uso de JavaScript: Se decidió utilizar JavaScript/TypeScript como lenguaje principal debido a su flexibilidad y facilidad de integración con herramientas como Jison. Además, permite desarrollar tanto el frontend como el backend dentro del mismo ecosistema tecnológico.

2. Uso de Jison para el análisis léxico y sintáctico: Se utilizó Jison para la generación automática del analizador léxico y sintáctico del lenguaje GoScript.

- Permite definir gramáticas de forma estructurada.
- Genera automáticamente el parser en JavaScript.
- Facilita la detección de errores léxicos y sintácticos.

3. Arquitectura cliente-servidor: Se implementó una separación entre frontend y backend para mejorar la organización del sistema.

- Separación de responsabilidades.
- Mayor mantenibilidad del código.
- Posibilidad de escalar o modificar cada parte de forma independiente.

4. Uso de Node.js en el backend: Se utilizó Node.js como entorno de ejecución del backend, lo que permite manejar peticiones HTTP y ejecutar el intérprete de forma eficiente.

5. Implementación basada en AST: El sistema construye un AST a partir del análisis sintáctico, el cual es recorrido para ejecutar el programa.

- Permite representar el código de forma estructurada.
- Facilita la ejecución y validación semántica.
- Mejora la organización de las instrucciones y expresiones.

6. Modelado del lenguaje mediante clases: Se implementaron distintas clases para representar instrucciones, expresiones y estructuras del lenguaje.

- Facilita la extensión del lenguaje.
- Permite una ejecución modular.
- Mejora la claridad del código.

Limitaciones del Programa

- 1. No existe concurrencia:** El sistema no soporta la ejecución de múltiples procesos o usuarios simultáneamente, ya que todo el análisis y ejecución se realiza de forma secuencial.
- 2. Validaciones semánticas limitadas:** El intérprete valida principalmente errores léxicos y sintácticos, así como algunos aspectos básicos en la ejecución, pero no incluye validaciones avanzadas como optimización de código o análisis profundo de tipos.
- 3. No hay optimización de ejecución:** El programa se ejecuta recorriendo el AST de manera directa, sin aplicar técnicas de optimización, lo que puede afectar el rendimiento en programas grandes.
- 4. Persistencia limitada:** El sistema no cuenta con un mecanismo de almacenamiento permanente, por lo que la información generada solo existe durante la ejecución del programa.
- 5. Dependencia del analizador generado:** El funcionamiento del sistema depende directamente de la gramática definida en Jison. Cualquier error o ambigüedad en la gramática puede afectar el análisis del lenguaje.
- 6. Interfaz básica:** La interfaz de usuario cumple con las funcionalidades esenciales (entrada de código y visualización de resultados), pero no incluye características avanzadas como autocompletado, resaltado de sintaxis o depuración.